

Simulazione della Diffusione del Calore 2D

Descrizione del problema

Implementare la parallelizzazione ibrida MPI+OpenMP dell'**equazione del calore 2D** su una griglia $N \times N$, con aggiornamento iterativo tramite stencil a 5 punti:

$$u_{\text{new}}[i][j] = (u[i-1][j] + u[i+1][j] + u[i][j-1] + u[i][j+1]) / 4.0$$

L'iterazione termina quando la norma L2 della differenza tra due iterazioni consecutive scende sotto una soglia ϵ (convergenza globale). Le condizioni al contorno sono di tipo Dirichlet con temperature diverse sui quattro lati (es. alto=100°, basso=0°, sinistra=75°, destra=25°), rendendo il problema fisicamente significativo e la soluzione non banale.

Implementazioni richieste

1. Versione sequenziale (heat_seq.c)

Baseline di riferimento per il calcolo di speedup ed efficienza.

2. Versione ibrida MPI+OpenMP (heat_hybrid.c)

- MPI_Init_thread con MPI_THREAD_FUNNELED
- Decomposizione del dominio per strisce di righe tra i processi MPI (MPI_Cart_create, MPI_Cart_shift)
- Halo exchange con **comunicazioni non bloccanti** (MPI_Isend/MPI_Irecv + MPI_Waitall) per sovrapporre il calcolo interno alla comunicazione (latency hiding)
- Calcolo dello stencil interno con `#pragma omp parallel for schedule(static)` mentre le comunicazioni sono in volo
- Completamento dell'halo (MPI_Waitall) e poi calcolo dei bordi
- Convergenza globale con MPI_Allreduce(MPI_SUM) sulla norma L2 locale

3. Versione MPI puro (heat_mpi.c)

Stessa struttura della ibrida ma senza OpenMP, usata per il confronto diretto a parità di core totali.

Analisi delle prestazioni

Variare N (es. 512, 1024, 2048), numero di processi MPI P e thread OpenMP T , mantenendo $P \times T = \text{costante}$ per il confronto ibrido vs MPI puro. Riportare:

- Tempo di esecuzione (wall time con MPI_Wtime)
- Speedup $S(p)$ ed efficienza $E(p)$ rispetto alla versione sequenziale
- **Strong scaling**: tempo al variare di $P \times T$ con N fisso ($P = \text{num. processi}$; $T = \text{num. thread}$)

- Confronto MPI puro vs ibrido a parità di core, ad es. con 8 core effettuare i seguenti test:
 - 8P × 1T
 - 4P × 2T
 - 2P × 4T
 - 1P × 8T

Deliverable

- Codice C/C++ commentato (`heat_seq.c`, `heat_hybrid.c`, `heat_mpi.c`) con `Makefile`
- Presentazione slides o relazione con: schema della decomposizione del dominio, pseudocodice del pattern latency hiding, tabelle e grafici di scaling, analisi dei risultati